

项目总体思路

一、研究目标

本项目的核心目标不是单纯检测视频中的物理错误，而是：

提升视频生成模型输出视频的物理一致性。

具体来说，首先训练或构建一个能够发现、定位和解释视频物理异常的 Physics Critic，然后将其接入本地部署的视频生成模型，形成“生成—检测—反馈—修正—再生成”的 Agentic Video Generation 闭环。

最终希望回答以下问题：

1. 现有视频生成模型最容易出现哪些物理错误；
2. Physics Critic 能否准确判断错误类型、异常对象和错误时序；
3. 细粒度物理反馈能否提高下一轮生成视频的物理一致性；
4. 多轮反馈是否比单次提示词增强更有效；
5. 物理性提升是否会损害画质、语义一致性和主体稳定性。

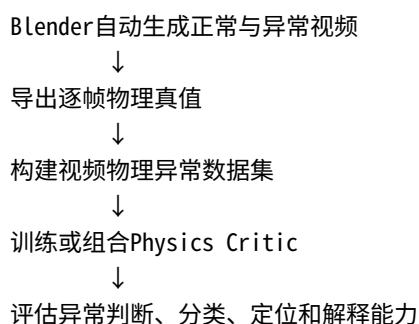
二、项目总体流程

项目可以分为三个主要阶段。

阶段一：Physics Critic 训练与评估

目标是得到一个可靠的视频物理异常检测模块。

整体流程：



Physics Critic 的输入包括：

- 视频；
- 原始生成提示词；
- Physics Planner 生成的预期物理过程；
- 可选的 Blender 参考仿真信息。

Physics Critic 的输出包括：

```
{
  "is_physical": false,
  "physics_score": 0.46,
  "violations": [
    {
      "object": "red_ball",
      "category": "premature_rebound",
      "start_frame": 47,
      "peak_frame": 49,
      "end_frame": 53,
      "critical_frames": [44, 47, 53],
      "reason": "The ball reverses direction before contacting the floor.",
      "repair_instruction": "Keep the ball moving downward until visible floor contact."
    }
  ]
}
```

Physics Critic 不应只是一个正常/异常二分类器，而应完成：

1. 物理合理性判断；
2. 异常类型识别；
3. 异常对象定位；
4. 异常开始帧和结束帧定位；
5. 关键证据帧选择；
6. 物理错误解释；
7. 修正建议生成。

阶段二：本地部署视频生成模型

计划本地部署 HunyuanVideo-1.5，将其作为 Agent 中的底层视频生成器。

推荐优先部署：

- 480p T2V；
- 480p I2V；
- 后期再考虑720p和超分辨率。

本地部署的主要优点：

- 没有按次生成额度；
- 可以反复调用；
- 能固定模型版本和随机种子；
- 便于批量实验；
- 方便接入 Python Agent；
- 后期可进行 LoRA 微调。

建议不要每次生成视频都重新启动模型，而是：

启动一次HunyuanVideo生成服务
 ↓
 模型保持加载
 ↓
 Agent通过Python接口或HTTP接口重复调用

生成接口至少需要控制：

```
{
  "prompt": "A red ball falls from a table.",
  "seed": 42,
  "resolution": "480p",
  "num_frames": 121,
  "num_inference_steps": 50,
  "image_path": null,
  "output_path": "outputs/video.mp4"
}
```

阶段三：Agentic Video Generation 闭环

在 Physics Critic 和 HunyuanVideo 都跑通后，搭建完整生成闭环：

用户提示词
 ↓
 Physics Planner
 ↓
 生成结构化物理计划
 ↓
 本地HunyuanVideo生成视频
 ↓
 Physics Critic检测物理问题
 ↓
 定位异常帧、对象、规律和原因
 ↓
 Repair Agent生成修正提示
 ↓
 重新调用HunyuanVideo
 ↓
 Selector选择历史最优视频

形式化表示为：

$$P_t \xrightarrow{G} V_t \xrightarrow{C} E_t \xrightarrow{R} P_{t+1}$$

其中：

- P_t ：第 t 轮提示词或物理计划；
- G ：视频生成器；
- V_t ：第 t 轮生成视频；
- C ：Physics Critic；
- E_t ：物理错误报告；
- R ：Repair Agent；
- P_{t+1} ：修正后的生成条件。

建议每轮不只生成一个视频，而是生成多个候选：

同一Prompt

- 多个随机种子
- 多个候选视频
- Physics Critic评分
- 选择物理性最高的视频

若最佳视频仍存在明显物理异常，再进入下一轮修正。

三、Blender MCP 和代码的作用

1. Blender MCP 的作用

Blender MCP 主要用于：

- 自动创建场景；
- 添加球体、墙壁、桌面、地面等物体；
- 设置质量、重力、摩擦和恢复系数；
- 设置刚体碰撞；
- 生成正常物理过程；
- 人为构造反向重力、穿透、提前反弹等异常；
- 执行物理仿真；
- 渲染视频；
- 导出逐帧物体状态；
- 导出异常开始帧和结束帧真值。

需要强调：

Blender MCP 不能直接读取 HunyuanVideo 生成视频中的真实物理参数。

它主要有三个作用：

1. 自动生成训练和评估数据；
2. 提供精确物理真值；
3. 根据 Physics Plan 生成简化参考仿真。

2. Codex或代码模块的作用

代码负责：

- 生成和修改 Blender Python 脚本；
- 批量生成数据；
- 读取视频；
- 抽帧；
- 检测和跟踪物体；
- 提取运动轨迹；
- 计算速度、加速度和轨迹变化；
- 检测碰撞、反弹、穿透和消失事件；
- 计算逐帧异常分数；
- 定位异常区间；
- 选择关键帧；
- 调用多模态模型复核；
- 控制完整 Agent loop。

四、Physics Critic 的建议结构

推荐采用混合式 Critic，而不是完全依赖一个大模型。

```
graph TD; A[视频] --> B[目标检测和分割]; B --> C[物体跟踪]; C --> D[轨迹和状态提取]; D --> E[事件检测]; E --> F[物理规则判断]; F --> G[Blender参考仿真对比]; G --> H[Video-VLM复核和解释]; H --> I[结构化物理错误报告];
```

具体模块包括：

```
Object Detector
Object Tracker
Trajectory Extractor
Event Detector
Physics Rule Engine
```

Temporal Localizer
Keyframe Selector
VLM Verifier
Result Fusion

五、如何定位异常时序和关键帧

首先对每个物体建立逐帧状态：

```
{  
  "frame": 48,  
  "object": "red_ball",  
  "center": [240, 160],  
  "bbox": [210, 130, 270, 190],  
  "visible": true,  
  "distance_to_floor": 18.5,  
  "overlap_with_floor": 0.0  
}
```

然后从轨迹中提取事件：

- 开始下落；
- 离开支撑面；
- 首次接触；
- 速度方向反转；
- 碰撞；
- 穿透；
- 消失；
- 再次出现；
- 反弹最高点。

异常时序通过比较“预期事件顺序”和“实际视觉事件顺序”得到。

例如预期：

下落
→ 接触地面
→ 反弹

实际检测：

下落
→ 反弹
→ 接触地面

则可判断为提前反弹。

关键帧建议选择：

F_pre: 异常前最后一个正常帧
F_start: 异常开始帧
F_peak: 异常最明显帧
F_post: 异常后结果帧

例如：

```
{  
  "start_frame": 47,  
  "peak_frame": 49,  
  "end_frame": 53,  
  "critical_frames": [44, 47, 49, 53]  
}
```

不要只依赖均匀抽帧。推荐：

先粗粒度抽帧寻找疑似时间段
→ 再对疑似区间逐帧分析
→ 最后选择证据关键帧

六、第一版建议研究的物理异常

第一阶段不宜覆盖过多类别，建议先做四类。

1. 重力异常

- 球向上掉落；
- 下落中突然悬浮；
- 无外力突然变向；
- 下落速度趋势明显异常。

2. 碰撞异常

- 接触前反弹；
- 接触后没有响应；
- 碰撞后异常加速；
- 碰撞方向明显错误。

3. 实体性与穿透

- 球穿墙；
- 球穿地面；
- 两物体互相穿透；
- 接触前发生碰撞响应。

4. 物体恒存

- 物体无缘无故消失；
- 遮挡后没有再次出现；
- 遮挡后身份或形状变化；
- 物体突然瞬移。

后续再扩展：

- 动量和能量；
- 流体；
- 光影；
- 多物体接触；
- 人体与物体交互。

七、数据集建议

每个样本建议保存：

```
sample_001/  
├── video.mp4  
├── scene.blend  
├── config.json  
├── trajectory.json  
├── contacts.json  
├── annotation.json  
└── keyframes/
```

标注格式：

```
{  
  "sample_id": "gravity_001",  
  "is_physical": false,  
  "violation_type": "reverse_gravity",  
  "object": "red_ball",  
  "start_frame": 38,  
  "peak_frame": 45,  
  "end_frame": 90,  
  "critical_frames": [35, 38, 45, 90],  
  "expected_rule": "unsupported objects accelerate downward",  
}
```



```
"repair_instruction": "Ensure continuous downward acceleration under gravity."  
}
```

数据来源建议包括：

1. Blender正常视频；
2. Blender人为异常视频；
3. HunyuanVideo真实生成错误；
4. 少量真实视频作为泛化测试。

Blender数据主要用于训练和精确标注，HunyuanVideo输出主要用于领域适配和闭环测试。

八、整体实验设计

需要比较以下方法：

```
Direct Generation  
Physics-aware Prompt  
Best-of-K  
LLM一次性Prompt改写  
Critic Feedback Loop  
LoRA + Critic Feedback Loop
```

其中 Proposed 方法为：

```
Physics Planner  
+ Best-of-K  
+ Physics Critic  
+ Repair Agent  
+ 多轮生成
```

主要评价指标包括：

Physics Critic 指标

- Accuracy;
- Macro-F1;
- 异常类型分类准确率;
- temporal IoU;
- 起始帧误差;
- 对象定位准确率;
- 关键帧命中率;
- 解释正确率。

整体 Agent 指标

- 最终物理一致性；
- 物理得分提升；
- 修正成功率；
- 错误减少率；
- 新增错误率；
- 语义保持率；
- 视觉质量变化；
- 平均循环次数；
- 生成时间和算力消耗。

最重要的研究问题之一是：

Physics Critic 的定位准确率和解释质量，是否会直接影响最终修复成功率。

九、需要准备的基础知识

1. Python基础

需要掌握：

- 函数和类；
- 文件读写；
- JSON；
- NumPy；
- 异常处理；
- 命令行参数；
- 虚拟环境；
- 多进程或任务队列基础。

2. Linux与环境管理

需要掌握：

- Ubuntu或其他Linux系统；
- Conda或venv；
- CUDA与PyTorch版本匹配；
- Git和GitHub；
- SSH；
- 后台运行；
- 日志查看；
- 磁盘和显存监控。

3. Blender基础

需要掌握：

- Blender界面基础；
- 对象、材质、灯光和摄像机；
- 时间轴和关键帧；
- 刚体和碰撞；
- 重力和摩擦；
- Blender Python API；
- 后台渲染；
- 导出MP4和JSON。

4. 计算机视觉基础

需要掌握：

- OpenCV；
- 视频读取和抽帧；
- 目标检测；
- 图像分割；
- 多目标跟踪；
- 光流；
- 轨迹平滑；
- 速度和加速度估计；
- 掩码距离和碰撞判断。

5. 深度学习基础

需要掌握：

- PyTorch；
- Dataset和DataLoader；
- 模型训练与验证；
- Loss和优化器；
- 混合精度；
- LoRA；
- 显存优化；
- 模型推理；
- 基本评价指标。

6. 多模态大模型基础

需要了解：

- 视频多模态模型输入方式；
- 均匀抽帧与动态抽帧；
- Prompt设计；
- 结构化JSON输出；
- 指令微调；

- VLM的幻觉和证据一致性问题。

7. 视频扩散模型基础

需要了解：

- Diffusion；
- DiT；
- VAE；
- 文生视频和图生视频；
- 推理步数；
- CFG；
- Seed；
- 模型卸载；
- Diffusers调用；
- LoRA微调。

8. Agent基础

需要掌握：

- Planner；
- Tool Calling；
- Critic；
- Repair；
- Selector；
- Loop Controller；
- 状态记录；
- 停止条件；
- 错误恢复机制。

十、硬件准备

推荐配置：

Linux系统
NVIDIA GPU
至少16GB显存，推荐24GB以上
至少32GB内存，推荐64GB
至少200GB可用硬盘

如果本地显存不足，可以先：

- 只做 Blender 数据；
- 训练轻量 Critic；
- 使用云GPU部署 HunyuanVideo；
- 或采用模型卸载和480p模式。

十一、代码工程建议

```
physgen/
├── dataset/
│   ├── blender_generator/
│   ├── annotations/
│   └── generated_videos/
├── critic/
│   ├── detector.py
│   ├── tracker.py
│   ├── trajectory.py
│   ├── event_detector.py
│   ├── physics_rules.py
│   ├── temporal_localizer.py
│   ├── keyframe_selector.py
│   └── vlm_verifier.py
├── generator/
│   ├── hunyuan_pipeline.py
│   ├── generator_server.py
│   └── schemas.py
├── agents/
│   ├── planner.py
│   ├── repairer.py
│   ├── selector.py
│   └── controller.py
├── evaluation/
│   ├── critic_metrics.py
│   ├── generation_metrics.py
│   └── human_evaluation.py
├── experiments/
│   ├── critic_benchmark.py
│   ├── feedback_ablation.py
│   └── end_to_end_loop.py
└── outputs/
```

十二、推荐实施顺序

第一步

用 Blender 生成一个正常的小球下落视频和一个提前反弹异常视频。

第二步

用 OpenCV 检测红球轨迹，定位：

- 接触帧；
- 方向反转帧；
- 异常区间；
- 关键帧。

第三步

扩展到：

- 穿地面；
- 穿墙；
- 突然消失；
- 反弹高度异常。

第四步

批量生成 Blender 数据集和JSON标注。

第五步

构建规则版 Physics Critic，并测试零样本视频多模态模型。

第六步

训练或微调学习型 Physics Critic。

第七步

本地部署 HunyuanVideo-1.5，完成单次视频生成。

第八步

封装本地生成服务，让 Python Agent 可重复调用。

第九步

跑通最小闭环：

Prompt
→ HunyuanVideo
→ Physics Critic
→ Repair Prompt
→ 再生成

第十步

增加 Best-of-K、多轮循环和整体消融实验。

十三、最终项目定位

本项目可以概括为：

利用 Blender 自动生成具有精确物理真值的视频数据，构建能够检测、定位和解释视频物理异常的混合 Physics Critic；随后将其接入本地部署的 HunyuanVideo-1.5，形成物理规划、视频生成、错误检测、反馈修正和候选选择闭环，从而增强生成视频的物理一致性。

真正的核心贡献不是简单使用 Blender MCP 或调用视频生成模型，而是：

1. 自动构建细粒度物理异常数据；
2. 准确定位异常时序和关键帧；
3. 将检测结果转换为可执行修正反馈；
4. 验证物理反馈对视频生成质量的实际提升。